

Kubernetes RBAC

Security Architecture Masterclass

*Authentication · Authorization · RBAC · Service Accounts · CRDs · Admission Controllers ·
Production Security*

9 Phases — Beginner to Enterprise Security Architecture

Phase 1: Foundations · Phase 2: RBAC Core · Phase 3: Practical Labs · Phase 4: Service
Accounts · Phase 5: Advanced RBAC

Phase 6: CRDs · Phase 7: RBAC + CRDs · Phase 8: Admission Controllers · Phase 9:
Production Architecture

Phase 1 — Security Foundations

Before we write a single line of YAML, we need to answer the most fundamental question: why does Kubernetes need security at all? The answer to this question — understood deeply — makes every RBAC rule you write feel intentional rather than mechanical.

1.1 The Kubernetes API Server — The Heart of Everything

Everything in Kubernetes — every `kubectl` command, every controller action, every pod creation — flows through a single central component: the kube-apiserver. This is not a minor implementation detail. It is the foundational security principle of Kubernetes architecture.

The kube-apiserver is a REST API server. Every Kubernetes operation is an HTTP request. When you run `'kubectl get pods'`, your `kubectl` binary sends an HTTP GET request to `https://kube-apiserver:6443/api/v1/namespaces/default/pods`. The API server processes this request, checks permissions, and returns the result.

This centralization is what makes Kubernetes security tractable. Instead of having to secure hundreds of individual components, you secure one gateway — the API server. Every request must pass through it. There are no backdoors.

1.2 The Airport Security Analogy — Authentication, Authorization, Admission

The kube-apiserver processes every incoming request through three sequential security layers. Think of it as airport security:

Security Layer	Airport Analogy and Kubernetes Reality
Authentication	Who are you? → Show your passport. In Kubernetes: prove your identity. The API server verifies you are who you claim to be using certificates, tokens, or OIDC. If authentication fails, you get HTTP 401 Unauthorized. The API server does not check what you want to do — just who you are.
Authorization	Are you allowed to be here? → Check your boarding pass matches your destination. In Kubernetes: RBAC checks whether your identity (user, service account, group) is permitted to perform the requested action (verb) on the requested resource (pod, secret, deployment) in the requested namespace. If denied, HTTP 403 Forbidden.
Admission Control	Final security check before boarding → Gate agent checks your passport photo matches your face, checks baggage weight, verifies no prohibited items. In Kubernetes: admission controllers can mutate requests (add default values, inject sidecars) or validate them (enforce policies, reject non-

Security Layer	Airport Analogy and Kubernetes Reality
	compliant resources). Happens AFTER authentication and authorization succeed.

The critical insight: These layers are sequential. A request that fails authentication never reaches authorization. A request that fails authorization never reaches admission. This layered defense is what makes the system robust.

1.3 What Happens Without RBAC — The Horror Story

Kubernetes RBAC is not enabled by default on ancient versions, and misconfigured RBAC in modern clusters is devastatingly dangerous. Let us trace a real attack scenario:

1. A developer deploys a containerized application with a misconfigured service account that has cluster-admin permissions (this is unfortunately common).
2. The application has a Remote Code Execution vulnerability — an attacker finds it and gains shell access inside the container.
3. Inside the container, the attacker discovers `/var/run/secrets/kubernetes.io/serviceaccount/token` — the service account token automatically mounted by Kubernetes.
4. The attacker uses this token: `kubectl --token=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token) get secrets --all-namespaces`
5. Because the service account has cluster-admin, the attacker reads every secret in every namespace — database passwords, API keys, TLS private keys, cloud credentials.
6. The attacker uses AWS credentials found in secrets to exfiltrate data from S3, create new IAM users for persistence, and pivot to the cloud infrastructure.
7. The entire cloud account is compromised from a single application vulnerability.

❑ This attack vector (service account token abuse) is one of the most common Kubernetes security incidents. The fix is least-privilege RBAC: service accounts should only have exactly the permissions they need for their function, and nothing more.

1.4 How kubectl Talks to Kubernetes

Understanding the request flow from your terminal to the cluster is essential for debugging RBAC issues.

```
kubectl get pods -n production

Step 1: kubectl reads ~/.kube/config
        Finds cluster endpoint, TLS CA cert, and credentials
        (certificate, token, or exec credentials plugin)

Step 2: kubectl constructs HTTP request:
        GET https://api.example.com:6443/api/v1/namespaces/production/pods
```

```

Authorization: Bearer <token> or TLS client certificate

Step 3: kube-apiserver receives request
├─ Authentication: verify token/cert → extract identity
│   Identity: {username: "hemant", groups: ["dev-team",
"system:authenticated"]}
├─ Authorization (RBAC): can hemant LIST pods in namespace production?
│   Check: does any RoleBinding/ClusterRoleBinding grant this?
│   YES → proceed NO → 403 Forbidden
├─ Admission: mutate/validate the request
│   (for GET requests, admission is skipped)

Step 4: etcd lookup → response returned to kubectl

```

1.5 Authentication Methods in Kubernetes

Method	Description and Use Cases
X.509 Client Certificates	The primary method for human admin access. The kubeconfig file contains a client certificate signed by the cluster's CA. The API server extracts the username from the CN field and groups from the O fields. kubectl uses this automatically.
Bearer Tokens (Static)	A static token file mounted into the API server. Simple but inflexible — tokens cannot be rotated without restarting API server. Used mainly in bootstrapping.
ServiceAccount Tokens	JWT tokens automatically created for every ServiceAccount. Mounted into pods at /var/run/secrets/kubernetes.io/serviceaccount/token. Used by applications running inside the cluster.
OIDC (OpenID Connect)	Integration with external identity providers (Google, Azure AD, Okta, Keycloak). The user authenticates with the IdP and receives a JWT. Kubernetes validates the JWT using the IdP's public keys. Industry standard for human user authentication in enterprise clusters.
Webhook Token Authentication	API server calls an external webhook to validate tokens. Flexible — allows integrating any authentication system.
Exec Credentials Plugin	kubectl calls an external binary to get credentials. Used for AWS EKS (aws eks get-token), GKE (gke-gcloud-auth-plugin), and Vault-based auth.

1.6 Interview Questions — Phase 1

Q: What are the three security layers in the Kubernetes API server request pipeline?

A: Authentication (who are you — verify identity), Authorization (what can you do — RBAC check), and Admission Control (should this request be allowed/modified — policy enforcement). They execute in this order. A request failing authentication never reaches authorization. This three-layer defense-in-depth is fundamental to Kubernetes security.

Q: A kubectl command returns '403 Forbidden'. What does this mean and how do you begin debugging?

A: 403 means authentication succeeded (identity was proven) but authorization failed (RBAC denied the action). The request was valid but the user/service account lacks the required permission. Debug: `kubectl auth can-i <verb> <resource> --as=<user> -n <namespace>` to test permissions. `kubectl describe rolebinding/clusterrolebinding` to find what's bound to the user. Check if the subject name in the binding matches exactly.

Phase 2 — RBAC Fundamentals

RBAC in Kubernetes is built on four objects: Role, ClusterRole, RoleBinding, and ClusterRoleBinding. Understanding how these four objects interact — and the crucial distinction between namespaced and cluster-wide scope — is the foundation of all Kubernetes security.

2.1 The Four RBAC Objects

Object	Scope and Purpose
Role	Namespaced. Defines a set of permissions (rules) that apply within a specific namespace. A Role in namespace 'production' cannot grant access to resources in 'staging'.
ClusterRole	Cluster-wide. Defines permissions that are not namespace-scoped. Used for: cluster-level resources (nodes, PVs, namespaces), non-resource URLs (/healthz, /metrics), or permissions that should apply across ALL namespaces.
RoleBinding	Namespaced. Binds a Role OR a ClusterRole to subjects (users, groups, service accounts) within a specific namespace. A RoleBinding of a ClusterRole restricts that ClusterRole's permissions to the binding's namespace only.
ClusterRoleBinding	Cluster-wide. Binds a ClusterRole to subjects. Grants permissions across ALL namespaces and cluster-level resources. The most powerful RBAC object — use with extreme caution.

The crucial ClusterRole + RoleBinding pattern

This is one of the most important and commonly misunderstood patterns: You can bind a ClusterRole with a RoleBinding. When you do this, the subject gets the ClusterRole's permissions but only within the namespace where the RoleBinding exists.

This pattern exists because you want to define permissions ONCE in a ClusterRole (e.g., a 'namespace-admin' role) and then reuse it across many namespaces by creating a RoleBinding in each namespace.

2.2 The Anatomy of a Rule — API Groups, Resources, and Verbs

Every permission in Kubernetes is expressed as a rule containing three components. Think of it as a sentence: 'Subject can VERB RESOURCE in APIGROUP.'

Verbs — What Actions Can Be Performed

Verb	HTTP Method and Meaning
get	HTTP GET on a specific resource. <code>kubectl get pod my-pod</code> . Reads one object by name.
list	HTTP GET on a resource collection. <code>kubectl get pods</code> . Reads all objects of a type.
watch	HTTP GET with <code>?watch=true</code> . <code>kubectl get pods -w</code> . Streams changes in real-time.
create	HTTP POST. <code>kubectl apply / kubectl create</code> . Creates a new resource.
update	HTTP PUT. Replaces an entire resource object. Full replacement.
patch	HTTP PATCH. <code>kubectl patch</code> . Modifies specific fields of a resource.
delete	HTTP DELETE. <code>kubectl delete</code> . Removes a resource.
deletecollection	HTTP DELETE on collection. <code>kubectl delete pods --all</code> . Bulk deletion.
exec	Special verb for pods/exec subresource. <code>kubectl exec</code> . Executes commands in containers.
portforward	Special verb for pods/portforward. <code>kubectl port-forward</code> .
proxy	Special verb for pods/proxy. HTTP proxying to pods.
log	Special verb for pods/log. <code>kubectl logs</code> .

API Groups — The Resource Namespace System

Kubernetes organizes resources into API groups. The API group determines the URL path where the resource is served and must match in RBAC rules.

API Group	Resources and Example URL
"" (core group)	Pods, services, configmaps, secrets, persistentvolumes, nodes. URL: <code>/api/v1/...</code>
apps	deployments, replicaset, statefulsets, daemonsets. URL: <code>/apis/apps/v1/...</code>
batch	jobs, cronjobs. URL: <code>/apis/batch/v1/...</code>
networking.k8s.io	ingresses, networkpolicies. URL: <code>/apis/networking.k8s.io/v1/...</code>
rbac.authorization.k8s.io	roles, clusterroles, rolebindings, clusterrolebindings.
storage.k8s.io	storageclasses, persistentvolumeclaims.
cert-manager.io	certificates, issuers, clusterissuers. (CRD example — Phase 6)

API Group	Resources and Example URL
Custom (CRDs)	Any string. e.g., monitoring.coreos.com for Prometheus CRDs.

Subresources — Actions Within Resources

Some Kubernetes resources have subresources that require explicit permission. This is critical for security — you might allow someone to view pods but not execute commands in them.

Subresource	Parent Resource and What It Allows
pods/exec	pods. Allows kubectl exec. Can execute arbitrary commands in running containers.
pods/log	pods. Allows kubectl logs. Can read container stdout/stderr.
pods/portforward	pods. Allows kubectl port-forward. Can tunnel to any port inside a pod.
pods/status	pods. Allows reading pod status. Used by controllers.
deployments/scale	deployments. Allows kubectl scale. Can change replica count.
nodes/proxy	nodes. Allows proxying requests to the kubelet API. Extremely sensitive.

❑ **pods/exec is one of the most dangerous permissions in Kubernetes. Anyone who can exec into a pod running as root can potentially escape the container, read host files, or access other pods' secrets. Never grant pods/exec without careful consideration.**

2.3 Role and ClusterRole — Full YAML Examples

Example 1: Read-Only Role for a Namespace

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: [""]          # core API group
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["services", "endpoints"]
  verbs: ["get", "list"]
```

Example 2: Deployment Manager Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
```



```

name: deployment-manager
namespace: production
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
# No access to secrets – intentional security boundary

```

Example 3: ClusterRole for Node Viewer

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-viewer
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["nodes/metrics"]
  verbs: ["get"]

```

2.4 RoleBinding and ClusterRoleBinding — Subjects

A binding connects a role to one or more subjects. Subjects can be users, groups, or service accounts.

Subjects explained

Subject Kind	Description and Format
User	A human user identity. In Kubernetes, users are not objects — they are strings extracted from the authentication credential. With X.509 certs, the CN field is the username. With OIDC, it comes from the token's sub or email claim. Format: kind: User, name: hemant@company.com
Group	A collection of users. With X.509 certs, groups come from the O (Organization) field in the certificate. With OIDC, groups come from a configured claim. Format: kind: Group, name: dev-team. The special group system:authenticated includes all authenticated users.
ServiceAccount	A Kubernetes object (unlike users). Belongs to a namespace. Used for application/workload identity. Format: kind:

Subject Kind	Description and Format
	ServiceAccount, name: my-service-account, namespace: production

RoleBinding — User to Namespace Role

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: hemant-pod-reader
  namespace: production
subjects:
- kind: User
  name: hemant@company.com # Must match exactly what authentication provides
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

RoleBinding — Group to ClusterRole (namespace-scoped)

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-team-namespace-admin
  namespace: production
subjects:
- kind: Group
  name: dev-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: admin # Built-in ClusterRole
  apiGroup: rbac.authorization.k8s.io
# Result: dev-team group has admin permissions in 'production' namespace only

```

ClusterRoleBinding — Cluster-Admin Binding

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: platform-team-cluster-admin
subjects:
- kind: Group
  name: platform-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin

```

```
apiGroup: rbac.authorization.k8s.io
```

❑ **cluster-admin ClusterRoleBinding gives COMPLETE, UNRESTRICTED access to everything in the cluster. This includes reading all secrets, modifying RBAC itself, accessing all namespaces, and managing nodes. Only the platform/SRE team should have this binding, and it should be GitOps-managed with audit logging.**

2.5 Built-in ClusterRoles

Built-in ClusterRole	Permissions Summary
cluster-admin	Full access to all resources in all namespaces. No restrictions. The highest privilege level.
admin	Full access to most resources in a namespace (via RoleBinding). Cannot modify namespace itself or resource quotas. Cannot modify RBAC.
edit	Can create, update, delete most resources. Cannot view or modify RBAC roles or bindings. Cannot access secrets by default in newer Kubernetes versions.
view	Read-only access to most resources. Cannot view secrets. Cannot modify anything.
system:node	Permissions needed by kubelet agents on each node. Can read pods, update node status.
system:kube-controller-manager	Permissions for the controller manager. Shows what controllers need to function.
system:aggregate-to-admin	Designed for aggregation — see Phase 5.

2.6 The Authorization Decision Flow

```
Request arrives at API server with identity:
  username: hemant@company.com
  groups: [dev-team, system:authenticated]
  namespace: production
  verb: delete
  resource: deployments

RBAC Decision Engine:
1. Collect all RoleBindings in namespace 'production'
   that reference subjects matching this identity
   (User hemant OR Group dev-team OR Group system:authenticated)

2. For each matching RoleBinding, get the referenced Role/ClusterRole
```

```
3. Check if any rule in any collected role permits:
  apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["delete"]

4. Also check all ClusterRoleBindings for the same identity

5. Decision: ALLOW if ANY rule matches
             DENY if NO rule matches

Result: ALLOW (if dev-team has deployment-manager role)
        DENY → 403 Forbidden (if no matching rule found)
```

□ *Kubernetes RBAC is purely additive. There is no DENY rule. If any matching rule allows the action, it is allowed. You cannot override an allow with a deny. This means the attack surface is defined entirely by what permissions exist — removing permissions is how you restrict access.*

2.7 Interview Questions — Phase 2

|Q: What is the difference between a Role and a ClusterRole?

A: A Role is namespace-scoped — it can only grant permissions to resources within its own namespace. A ClusterRole is cluster-scoped — it can grant permissions to cluster-level resources (nodes, PVs, namespaces) or permissions that span all namespaces. Importantly, a ClusterRole can be bound with a RoleBinding to limit its scope to a single namespace, or with a ClusterRoleBinding to grant permissions cluster-wide.

|Q: Explain the subjects field in a RoleBinding. What can it contain?

A: Subjects are the entities receiving permissions. Three kinds: User (a human user identity string, extracted from the authentication credential — not a Kubernetes object), Group (a collection of users from auth credentials), and ServiceAccount (a Kubernetes namespace-scoped object for application identity). Multiple subjects can share a single binding. The name field must match exactly what the authentication layer provides.

|Q: Can a RoleBinding reference a ClusterRole? What is the effect?

A: Yes. When a RoleBinding references a ClusterRole, the subjects receive the ClusterRole's permissions but ONLY within the namespace where the RoleBinding exists. This is a key pattern for permission reuse — define permissions once in a ClusterRole (e.g., 'namespace-admin'), then create RoleBindings in each namespace to apply those permissions locally without granting cluster-wide access.

Phase 3 — Practical RBAC Labs

Theory becomes skill through practice. This phase provides complete, working RBAC configurations for common real-world scenarios.

3.1 Lab: Read-Only User for a Namespace

Scenario: A junior developer needs to view pods, deployments, and services in the 'staging' namespace but should not be able to modify anything or read secrets.

```
# Role - read-only access to common resources
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: readonly-developer
  namespace: staging
rules:
- apiGroups: [""]
  resources:
    - pods
    - pods/log
    - services
    - endpoints
    - configmaps
    - events
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources:
    - deployments
    - replicaset
    - statefulsets
    - daemonsets
  verbs: ["get", "list", "watch"]
- apiGroups: ["batch"]
  resources: ["jobs", "cronjobs"]
  verbs: ["get", "list", "watch"]
# Secrets intentionally omitted
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: junior-dev-readonly
  namespace: staging
subjects:
- kind: User
  name: junior@company.com
  apiGroup: rbac.authorization.k8s.io
roleRef:
```

```
kind: Role
name: readonly-developer
apiGroup: rbac.authorization.k8s.io
```

Validation commands:

```
# Verify the permissions
kubectl auth can-i get pods -n staging --as=junior@company.com
# yes

kubectl auth can-i delete pods -n staging --as=junior@company.com
# no

kubectl auth can-i get secrets -n staging --as=junior@company.com
# no

# Full permission check
kubectl auth can-i --list -n staging --as=junior@company.com
```

3.2 Lab: CI/CD Service Account

Scenario: A Jenkins/GitHub Actions pipeline needs to deploy to the 'production' namespace. It needs to create and update deployments, manage configmaps, but must NOT access secrets or other namespaces.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cicd-deployer
  namespace: production
automountServiceAccountToken: false # Disable auto-mount - we'll mount explicitly
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cicd-deploy-role
  namespace: production
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["pods"]
```

```
  verbs: ["get", "list", "watch"]
# No secrets access – CI/CD pipeline should not read secrets
# No delete – prevent accidental production deletion
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cicd-deployer-binding
  namespace: production
subjects:
- kind: ServiceAccount
  name: cicd-deployer
  namespace: production
roleRef:
  kind: Role
  name: cicd-deploy-role
apiGroup: rbac.authorization.k8s.io
```

Creating and using the CI/CD token

```
# Create a long-lived token for CI/CD system
# (Kubernetes 1.24+ requires explicit secret creation for long-lived tokens)
kubectl create token cicd-deployer -n production --duration=8760h

# Or create a Secret-based token for permanent use:
kubectl apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: cicd-deployer-token
  namespace: production
  annotations:
    kubernetes.io/service-account.name: cicd-deployer
type: kubernetes.io/service-account-token
EOF

# Get the token
kubectl get secret cicd-deployer-token -n production \
  -o jsonpath='{.data.token}' | base64 -d
```

3.3 Debugging RBAC Failures

kubectl auth can-i — Your First Debug Tool

```
# Basic check: can the current user do something?
kubectl auth can-i create deployments -n production

# Check as a specific user (requires impersonation permission)
kubectl auth can-i get secrets -n production --as=developer@company.com
```

```
# Check as a service account
kubectl auth can-i list pods -n production \
  --as=system:serviceaccount:production:cicd-deployer

# List ALL permissions for a user in a namespace
kubectl auth can-i --list -n production \
  --as=system:serviceaccount:production:cicd-deployer

# List ALL cluster-level permissions
kubectl auth can-i --list \
  --as=system:serviceaccount:production:cicd-deployer
```

Finding What is Bound to a Subject

```
# Find all RoleBindings in a namespace
kubectl get rolebindings -n production -o yaml

# Find all ClusterRoleBindings for a user
kubectl get clusterrolebindings -o json | \
  jq '.items[] | select(.subjects[0].name=="developer@company.com")'

# Describe a specific binding
kubectl describe rolebinding cicd-deployer-binding -n production

# Find what role grants a specific permission
kubectl get roles -n production -o yaml | grep -A5 'secrets'
```

Common RBAC Mistakes and Fixes

Mistake	Explanation and Fix
Subject name mismatch	The most common RBAC failure. The name in the RoleBinding subject must match EXACTLY what authentication provides. With OIDC, check the token's claims. With certs, check the CN field: <code>openssl x509 -in client.crt -noout -subject</code>
Wrong namespace in ServiceAccount subject	ServiceAccount subjects MUST include the namespace field. <code>kind: ServiceAccount, name: my-sa, namespace: my-ns</code> . Missing namespace causes binding to fail silently.
Using Role for cluster resources	Nodes, PVs, namespaces, ClusterRoles are cluster-scoped. They cannot be accessed by a Role — you need a ClusterRole bound with ClusterRoleBinding.
Forgetting API group	Rules for deployments must use <code>apiGroups: ["apps"]</code> . Rules for pods use <code>apiGroups: [""]</code> . Wrong API group means the rule is never matched.
No access to events	Events are in the core API group. Many dashboards need <code>get/list/watch</code> on events. Frequently forgotten.

Phase 4 — Service Accounts and Pod Identity

Service accounts are the identity mechanism for workloads (pods) running inside Kubernetes. Understanding them is critical because they are the most common vector for privilege escalation and lateral movement in Kubernetes security incidents.

4.1 What is a Service Account?

In the real world, when a company's HR software needs to query the employee database, it does not log in as a human user. It has its own system identity — a service account with only the specific permissions it needs. Kubernetes service accounts serve the same purpose.

Every pod in Kubernetes has a service account. If you do not specify one, the pod gets the 'default' service account in its namespace. The service account's identity is expressed as a JWT token that is automatically mounted into the pod at a well-known path.

Property	Human User vs Service Account
Kubernetes Object	Human: NOT a Kubernetes object — just a string identity from auth. ServiceAccount: IS a Kubernetes object, namespace-scoped.
Creation	Human: managed by external auth system (OIDC, certs). ServiceAccount: <code>kubectl create serviceaccount</code> or YAML manifest.
Token	Human: kubeconfig file on developer's machine. ServiceAccount: JWT token mounted into pod filesystem.
Namespace	Human: not namespace-scoped. ServiceAccount: always belongs to a specific namespace.
Rotation	Human: managed by IdP or certificate authority. ServiceAccount: Kubernetes rotates projected tokens automatically (default 1 hour TTL).

4.2 Service Account Token Mounting

Default behavior (before Kubernetes 1.24)

Every pod automatically gets a service account token mounted at `/var/run/secrets/kubernetes.io/serviceaccount/`. This directory contains three files:

- `token` — a JWT signed by the cluster's CA, identifying the service account
- `ca.crt` — the cluster's CA certificate (for validating API server TLS)
- `namespace` — the pod's namespace

Projected Tokens (Kubernetes 1.20+, default in 1.22+)

Projected service account tokens are a significant security improvement. They are audience-bound (only valid for a specific audience), time-bound (expire after a configurable duration, default 1 hour), and are automatically rotated by Kubernetes before expiry.

```
# Pod spec with explicit projected token configuration
apiVersion: v1
kind: Pod
metadata:
  name: my-secure-pod
spec:
  serviceAccountName: my-service-account
  volumes:
  - name: token-volume
    projected:
      sources:
      - serviceAccountToken:
          path: token
          expirationSeconds: 3600    # 1 hour token
          audience: my-application  # audience-bound
  containers:
  - name: app
    image: my-app:latest
    volumeMounts:
    - mountPath: /var/run/secrets/tokens
      name: token-volume
```

Disabling Auto-Mount — Least Privilege

Most application pods do not need to talk to the Kubernetes API. For these pods, mounting the service account token creates an unnecessary attack surface — if the pod is compromised, the attacker gets a valid token.

```
# Disable token auto-mount at ServiceAccount level
apiVersion: v1
kind: ServiceAccount
metadata:
  name: no-api-access
  namespace: production
automountServiceAccountToken: false

# Or disable at pod level
spec:
  automountServiceAccountToken: false
```

✓ Best practice: Set `automountServiceAccountToken: false` on service accounts for all applications that do not need Kubernetes API access. This is most application workloads.

4.3 IRSA — Service Accounts for Cloud Provider Authentication

IRSA (IAM Roles for Service Accounts) on EKS extends Kubernetes service account identity to AWS IAM. Instead of attaching IAM roles to EC2 nodes (which grants those permissions to ALL pods on the node), IRSA allows individual pods to assume specific IAM roles.

How it works: The EKS cluster has an OIDC provider. The pod's projected service account token is used as a web identity token. The AWS SDK calls `sts:AssumeRoleWithWebIdentity` using this token to get temporary credentials for the specific IAM role.

```
# Service account annotated with IAM role
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ebs-csi-controller-sa
  namespace: kube-system
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::051556733177:role/EBS-CSI-Driver-Role
```

```
# The pod using this SA automatically gets AWS credentials
# scoped to the annotated IAM role - no access keys needed
```

4.4 Interview Questions — Phase 4

Q: What is `automountServiceAccountToken` and why should you disable it for most pods?

A: `automountServiceAccountToken` controls whether Kubernetes automatically mounts the service account JWT token into the pod filesystem. When enabled (the default), every pod gets a valid token that can make Kubernetes API calls. For application pods that do not need to interact with the Kubernetes API (the vast majority of workloads), this token is unnecessary attack surface. If an attacker compromises the pod, they get a valid token. Disabling it eliminates this exposure. Enable it only for pods that explicitly need API access (operators, monitoring agents, CI/CD pods).

Q: Explain the difference between a long-lived service account token (Secret-based) and a projected service account token.

A: Secret-based tokens (pre-1.24 default) are permanent — they never expire and are valid until the secret is deleted. This creates security risk: if exfiltrated, they work indefinitely. Projected tokens (1.22+ default) are automatically generated, have a configurable TTL (default 1 hour), are audience-bound (only valid for the intended recipient), and are automatically rotated by Kubernetes before expiry. Projected tokens are significantly more secure — exfiltrated tokens expire quickly.

Phase 5 — Advanced RBAC

This phase covers the security implications and attack paths that senior engineers must understand. Misconfigured RBAC is the most common path to Kubernetes cluster compromise.

5.1 Aggregated ClusterRoles

Kubernetes has a mechanism for automatically combining multiple ClusterRoles into one aggregate role. This is used extensively in the built-in admin, edit, and view roles. When a ClusterRole has specific aggregation labels, Kubernetes automatically merges all matching ClusterRoles into it.

```
# Define the aggregate ClusterRole with a selector
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: monitoring-viewer
  labels:
    rbac.authorization.k8s.io/aggregate-to-view: "true"
rules:
- apiGroups: ["monitoring.coreos.com"]
  resources: ["prometheusrules", "servicemonitors"]
  verbs: ["get", "list", "watch"]

# Now any user with the built-in 'view' ClusterRole ALSO gets
# these monitoring permissions automatically — without modifying
# the built-in view role directly
```

5.2 Privilege Escalation Risks

The most dangerous RBAC misconfigurations are those that allow privilege escalation — gaining more permissions than intended. Kubernetes has protections, but many dangerous patterns exist.

Escalation Path 1: Permission to create RoleBindings

❑ If a user can create RoleBindings, they can grant themselves any Role or ClusterRole that already exists — including cluster-admin. This is equivalent to having cluster-admin.

```
# Dangerous: this role allows self-escalation
rules:
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["rolebindings"]
  verbs: ["create"]
# A user with this can: kubectl create rolebinding my-admin-binding
# --clusterrole=cluster-admin --user=myself -n production
```

Escalation Path 2: Permission to update Roles

- ❑ If a user can update or patch existing Roles, they can add any permission to that role — including modifying their own role to give themselves more access.

Escalation Path 3: Permission to exec into privileged pods

If a user can exec into a pod running as root with access to the host network or host PID, they can effectively escape the container and access the host node — which has kubelet credentials for full cluster control.

Escalation Path 4: Secrets read access

Reading secrets in kube-system can give access to: bootstrap tokens (can be used to add new nodes), service account tokens for system components, and in some configurations, cloud provider credentials.

Kubernetes protections against escalation

Kubernetes has a built-in escalation prevention: you cannot grant permissions you do not already have. If you try to create a Role or RoleBinding that grants more permissions than you currently have, the API server rejects it with 'cannot escalate'. This prevents a user with 'edit' on roles from granting themselves cluster-admin. Exception: users with the special verb 'escalate' on roles can bypass this.

5.3 Wildcard Dangers

```
# EXTREMELY DANGEROUS — Never use in production
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
# This is equivalent to cluster-admin
```

- ❑ Wildcards (*) in RBAC rules match everything — including future resources and API groups you have not even installed yet. Wildcards in production RBAC are a severe security risk and should trigger an immediate security review.

5.4 Secret Access Implications

Kubernetes secrets are the most sensitive resources in a cluster. Understanding who can access secrets — and through what paths — is critical.

Secret Access Path	Risk Level and Explanation
Direct: get secrets	HIGH. Can read any secret value in the namespace. Gives access to database passwords, API keys, TLS private keys.

Secret Access Path	Risk Level and Explanation
Create pods/deployments	HIGH. A user who can create pods can mount any secret as a volume or environment variable — effectively reading it. This is often overlooked.
Exec into pods	HIGH. Can read environment variables (which often contain secrets) and mounted secret files from inside the container.
Get pods	MEDIUM. Reveals which secrets are mounted (by name) even if you cannot read their values.
Watch secrets	HIGH. Same as get — gets the full secret value on creation/update events.

□ A user who can create pods in a namespace can effectively read all secrets in that namespace by creating a pod that mounts the secret and then exec'ing into it. The ability to create pods is therefore nearly equivalent to the ability to read secrets.

5.5 Interview Questions — Phase 5

|Q: What is RBAC privilege escalation in Kubernetes and how does Kubernetes prevent it?

A: Privilege escalation is gaining more permissions than currently held by manipulating RBAC objects. Example: a user with 'create rolebindings' permission can bind themselves to cluster-admin. Kubernetes prevents escalation by blocking users from creating Roles or RoleBindings that grant permissions they do not already possess (without the 'escalate' verb). Mitigations: use least-privilege, never grant create/update on rbac resources to non-admins, review all RBAC changes in GitOps workflows.

|Q: Why is the ability to create pods nearly equivalent to secrets read access?

A: A user who can create pods can specify any secret as a volume mount or environment variable in the pod spec. They can then exec into the pod (if they have pods/exec) or read the pod's logs to access the secret values. Even without exec, they can mount a secret at a path and configure the container to print its contents. Secret access control must therefore consider indirect access paths, not just direct 'get secrets' permission.

Phase 6 — Custom Resource Definitions (CRDs)

CRDs are one of the most powerful and frequently used extensibility mechanisms in Kubernetes. Almost every major Kubernetes tool you use — Prometheus, ArgoCD, cert-manager, Istio, Crossplane — works by installing CRDs and a controller. Understanding CRDs deeply makes you a platform engineer, not just a Kubernetes user.

6.1 Why CRDs Exist — The Kubernetes Extension Philosophy

Kubernetes was designed from the beginning to be extensible. The core team knew they could not anticipate every use case. Rather than building a monolithic system, they built a platform that could be extended with new resource types.

Think of Kubernetes as an operating system. The OS provides core primitives: processes, files, networking. Applications are built on top of these primitives. Kubernetes provides core primitives: pods, services, configmaps. CRDs are like installing new applications that extend the OS with new capabilities.

Before CRDs, extending Kubernetes required forking the source code or using ConfigMaps as a database (the infamous ConfigMap abuse pattern). CRDs provide a first-class mechanism for adding new resource types that behave exactly like built-in resources: stored in etcd, served by the API server, watched by controllers, and managed by kubectl.

6.2 The CRD Ecosystem — What CRDs Enable

Tool	CRDs It Installs and Their Purpose
cert-manager	Certificate, CertificateRequest, Issuer, ClusterIssuer, CertificateRevocationList. Automates TLS certificate lifecycle.
ArgoCD	Application, AppProject, ApplicationSet. Manages GitOps deployment pipelines.
Prometheus Operator	Prometheus, ServiceMonitor, PodMonitor, PrometheusRule, AlertManager. Manages monitoring infrastructure.
Istio	VirtualService, DestinationRule, Gateway, ServiceEntry, PeerAuthentication. Service mesh configuration.
Crossplane	Provider, ProviderConfig, CompositeResourceDefinition. Infrastructure provisioning via Kubernetes.
Argo Rollouts	Rollout, AnalysisTemplate, AnalysisRun. Progressive delivery strategies.
Velero	Backup, Restore, Schedule, VolumeSnapshotLocation. Cluster backup and disaster recovery.

6.3 CRD vs CR — Understanding the Distinction

This is commonly confused. There are two separate concepts:

- CRD (CustomResourceDefinition) — The schema definition. Tells Kubernetes 'a new resource type called Certificate exists, with these fields, this validation, this scope.' A CRD is like a database schema or a class definition. You install ONE CRD per resource type.
- CR (Custom Resource) — An instance of the CRD. Once the Certificate CRD is installed, you can create many Certificate objects. These are custom resources. Like rows in a database table, or objects instantiated from a class.

6.4 Anatomy of a CRD — Full YAML

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: tenants.tenantops.io # Must be: <plural>.<group>
spec:
  group: tenantops.io # API group — appears in URL and RBAC
  names:
    kind: Tenant # Capital camel case — used in YAML kind field
    plural: tenants # URL path: /apis/tenantops.io/v1/tenants
    singular: tenant # CLI: kubectl get tenant
    shortNames: [ten] # Short alias: kubectl get ten
  scope: Namespaced # Namespaced or Cluster
  versions:
    - name: v1
      served: true # API server serves this version
      storage: true # This version stored in etcd
      schema:
        openAPIV3Schema: # Validation schema
          type: object
          properties:
            spec:
              type: object
              required: [companyName, plan]
              properties:
                companyName:
                  type: string
                  maxLength: 100
                plan:
                  type: string
                  enum: [starter, professional, enterprise]
                maxServers:
                  type: integer
                  minimum: 1
                  maximum: 1000
            status:
              type: object
              properties:
                phase:
```

```

        type: string
    provisionedAt:
        type: string
    subresources:
        status: {}          # Enable /status subresource
    additionalPrinterColumns:
    - name: Company
      type: string
      jsonPath: .spec.companyName
    - name: Plan
      type: string
      jsonPath: .spec.plan
    - name: Phase
      type: string
      jsonPath: .status.phase

```

6.5 Creating a Custom Resource

```

apiVersion: tenantops.io/v1
kind: Tenant
metadata:
  name: acme-corp
  namespace: tenants
spec:
  companyName: ACME Corporation
  plan: enterprise
  maxServers: 500
status:
  phase: Provisioning

```

```

# kubectl treats this like any built-in resource
kubectl get tenants -n tenants
kubectl describe tenant acme-corp -n tenants
kubectl get ten -A # using short name

```

6.6 The Operator Pattern — Controller + CRD

A CRD alone is just storage — etcd with a schema. The power comes from a controller (often called an Operator) that watches for custom resources and takes action to reconcile the desired state with the actual state.

The Operator pattern is the Kubernetes extension mechanism for encoding operational knowledge. An Operator knows how to: provision resources when a CR is created, update them when the CR changes, and clean up when the CR is deleted.

```

Operator Reconciliation Loop:

WATCH: API server streams events for Tenant objects

```

```
ON CREATE Tenant 'acme-corp':
  → Create namespace 'tenant-acme-corp'
  → Create ResourceQuota for the plan limits
  → Create default RBAC roles for the tenant
  → Create NetworkPolicy for isolation
  → Update Tenant status.phase = 'Active'

ON UPDATE Tenant 'acme-corp' (plan: enterprise → starter):
  → Update ResourceQuota to reduce limits
  → Update Tenant status.phase = 'Updated'

ON DELETE Tenant 'acme-corp':
  → Remove namespace (all resources deleted)
  → Clean up any cluster-level resources
```

6.7 Interview Questions — Phase 6

Q: What is the difference between a CRD and a Custom Resource?

A: A CRD (CustomResourceDefinition) is the schema definition — it tells Kubernetes that a new resource type exists, with its fields, validation rules, scope (namespaced vs cluster), and API group. It is installed once. A Custom Resource is an instance of that type — an actual object stored in etcd. The CRD is the class; the CR is the object. For example, the Certificate CRD defines what a cert-manager Certificate looks like; each individual TLS certificate you create is a Custom Resource of kind Certificate.

Q: Explain the Operator pattern in Kubernetes.

A: An Operator combines a CRD (defining the desired state schema) with a controller (a running process that watches for CRs and reconciles actual state to desired state). The controller implements a reconciliation loop: watch for changes to custom resources → compare desired state to actual state → take actions to eliminate the difference. Operators encode operational knowledge into code. Instead of a human running kubectl commands to manage a complex stateful application (like a database cluster), the Operator does it automatically in response to simple declarative custom resources.

Phase 7 — RBAC for CRDs and Operators

When you install CRDs, you need RBAC rules to control who can create, read, update, and delete those custom resources. This is where RBAC and CRDs come together — and where many operators get misconfigured with overly broad permissions.

7.1 RBAC Rules for CRDs

CRD resources use the CRD's API group in RBAC rules. The API group is the group field from the CRD spec.

```
# Allow users to manage cert-manager Certificates
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cert-manager-user
  namespace: production
rules:
- apiGroups: ["cert-manager.io"]    # CRD group
  resources:
    - certificates
    - certificaterequests
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: ["cert-manager.io"]
  resources: ["issuers"]
  verbs: ["get", "list", "watch"]
# ClusterIssuers are cluster-scoped — need ClusterRole for those
```

```
# Allow users to manage ArgoCD applications
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: argocd-app-manager
  namespace: argocd
rules:
- apiGroups: ["argoproj.io"]
  resources: ["applications", "appprojects"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
```

7.2 Operator Permissions — Why They Need Elevated Access

Kubernetes Operators need broad permissions because they provision and manage Kubernetes resources on behalf of users. A database operator, for example, needs to create StatefulSets, Services, ConfigMaps, Secrets, and PVCs. A networking operator needs to create NetworkPolicies.

The tension: Operators need broad permissions to do their job, but broad permissions are a security risk. If an operator is compromised (e.g., a supply chain attack on the operator's container image), its service account could be used to compromise the entire cluster.

```
# Example: Prometheus Operator service account permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus-operator
rules:
- apiGroups: ["monitoring.coreos.com"]
  resources: ["*"]
  verbs: ["*"]
- apiGroups: [""]
  resources:
    - configmaps
    - secrets
    - services
    - serviceaccounts
  verbs: ["*"]
- apiGroups: ["apps"]
  resources: ["statefulsets"]
  verbs: ["*"]
# This is broad – necessary for operator function
# Mitigated by: operator runs in dedicated namespace,
#               regular image security scanning,
#               network policies limiting operator's network access
```

7.3 Namespace-Scoped vs Cluster-Scoped CRDs

CRDs can be either namespaced (scope: Namespaced) or cluster-scoped (scope: Cluster). This affects RBAC:

- Namespaced CRDs: Resources are created within namespaces. RBAC Roles can control access per-namespace. Example: cert-manager Certificate is namespaced.
- Cluster-scoped CRDs: Resources exist at cluster level, not in any namespace. Require ClusterRole/ClusterRoleBinding for access. Example: cert-manager ClusterIssuer is cluster-scoped.

```
# For cluster-scoped CRDs, you need a ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: clusterissuer-viewer
rules:
- apiGroups: ["cert-manager.io"]
  resources: ["clusterissuers"] # Cluster-scoped CRD
  verbs: ["get", "list", "watch"]
```

7.4 Interview Questions — Phase 7

|Q: How do you write RBAC rules for a Custom Resource? What goes in apiGroups?

A: The apiGroups field in RBAC rules must match the group field from the CRD definition. For example, if a CRD has group: monitoring.coreos.com, then RBAC rules for those resources use apiGroups: ['monitoring.coreos.com']. The resources field uses the plural name from the CRD. The verbs are the same standard verbs as built-in resources. This is the same pattern as built-in resources — core resources use apiGroups: [''] and apps resources use apiGroups: ['apps'].

Phase 8 — Admission Controllers and Policy

Authentication says 'who are you?' Authorization says 'are you allowed?' Admission control says 'should this specific request, with these specific parameters, actually be allowed or modified?' Admission control is your last line of defense before resources are persisted to etcd.

8.1 What Admission Controllers Do

Admission controllers intercept API server requests after authentication and authorization but before the object is written to etcd. There are two types:

- **Mutating Admission Controllers** — can modify (mutate) the request object before it is stored. Example: automatically injecting a sidecar container into every pod, adding default labels, setting default resource limits.
- **Validating Admission Controllers** — can only allow or deny the request. Cannot modify. Example: rejecting pods without resource limits, requiring specific labels, enforcing image registry policies.

Mutating webhooks run FIRST, then validating webhooks. This allows mutations to happen (e.g., adding defaults) before validation checks those defaults.

8.2 Built-in Admission Controllers

Controller	What It Does
NamespaceLifecycle	Rejects requests to create resources in terminating namespaces. Prevents resource creation in nonexistent namespaces.
LimitRanger	Enforces resource limits and requests. Applies defaults from LimitRange objects.
ResourceQuota	Enforces ResourceQuota objects — rejects resource creation that would exceed quota limits.
PodSecurity	Enforces Pod Security Standards (restricted, baseline, privileged) at the namespace level. Replaced PodSecurityPolicy (deprecated in 1.21, removed in 1.25).
ServiceAccount	Automatically adds service account tokens to pods. Sets default service account if not specified.
MutatingAdmissionWebhook	Calls external webhooks that can mutate objects. The basis for Istio sidecar injection, cert-manager injection, etc.
ValidatingAdmissionWebhook	Calls external webhooks that validate objects. The basis for OPA Gatekeeper, Kyverno, etc.

8.3 Webhook-Based Admission — Mutating Example

```
# MutatingWebhookConfiguration — Istio sidecar injection pattern
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: istio-sidecar-injector
webhooks:
- name: sidecar-injector.istio.io
  rules:
  - apiGroups: [""]
    apiVersions: ["v1"]
    operations: ["CREATE"]
    resources: ["pods"]
  namespaceSelector:
    matchLabels:
      istio-injection: enabled # Only inject in labeled namespaces
  clientConfig:
    service:
      name: istiod
      namespace: istio-system
      path: /inject
  admissionReviewVersions: ["v1"]
  sideEffects: None
  failurePolicy: Fail # Reject pod if webhook is unavailable
```

8.4 OPA Gatekeeper — Policy as Code

OPA (Open Policy Agent) Gatekeeper is a validating admission webhook that enforces policies written in Rego (a policy language). It introduces two CRDs: ConstraintTemplate (defines the policy) and Constraint (applies the policy with parameters).

ConstraintTemplate — Requiring Resource Limits

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: requireresourcelimits
spec:
  crd:
    spec:
      names:
        kind: RequireResourceLimits
  targets:
  - target: admission.k8s.gatekeeper.sh
    rego: |
      package requireresourcelimits
      violation[{"msg": msg}] {
        container := input.review.object.spec.containers[_]
        not container.resources.limits.cpu
```



```

    msg := sprintf("Container '%v' must set CPU limits",
                    [container.name])
}
violation[{"msg": msg}] {
    container := input.review.object.spec.containers[_]
    not container.resources.limits.memory
    msg := sprintf("Container '%v' must set memory limits",
                    [container.name])
}

```

```

# Constraint - apply the policy to a namespace
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: RequireResourceLimits
metadata:
  name: require-resource-limits-production
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces: ["production", "staging"]

```

8.5 Kyverno — Kubernetes-Native Policy

Kyverno is an alternative to OPA Gatekeeper that uses Kubernetes YAML syntax for policies instead of Rego. It is simpler to learn for Kubernetes practitioners and supports both validation and mutation.

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-pod-labels
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: check-team-label
      match:
        any:
          - resources:
              kinds: [Pod]
              namespaces: [production, staging]
      validate:
        message: "The label 'team' is required on all production pods."
        pattern:
          metadata:
            labels:
              team: "?*" # Must exist and be non-empty

```

8.6 Interview Questions — Phase 8

Q: What is the difference between a mutating and validating admission webhook?

A: Mutating webhooks execute first and can modify the incoming request object — adding annotations, injecting containers, setting defaults. Validating webhooks execute after mutating webhooks and can only allow or deny the request — they cannot modify it. Both receive an AdmissionReview object containing the request details and must return an AdmissionResponse indicating allow/deny, with optional mutation (for mutating) or reason (for validating).

Q: What does failurePolicy: Fail vs failurePolicy: Ignore mean on an admission webhook?

A: failurePolicy: Fail means if the webhook is unavailable (network timeout, service crashed), the API server rejects the request. This ensures policy is always enforced but risks cluster disruption if the webhook is down. failurePolicy: Ignore means if the webhook is unavailable, the request proceeds as if the webhook approved it. This avoids disruption but creates a window where policy is not enforced. For security-critical policies (e.g., blocking privileged containers), use Fail. For convenience mutations, Ignore may be acceptable.

Phase 9 — Production Security Architecture

This phase brings everything together into an actionable enterprise security architecture. These are the patterns used by mature, security-conscious organizations running Kubernetes in production.

9.1 Enterprise RBAC Design Principles

Production RBAC design follows several key principles that should guide every decision:

8. Least Privilege — every subject (user, service account) gets exactly the permissions needed for their function, and nothing more. Start with nothing and add permissions as needed, never remove from a large set.
9. Separation of Duties — different roles for different responsibilities. Platform team manages cluster infrastructure. Application teams manage their namespaces. Security team reviews RBAC changes. No single team has unrestricted access.
10. GitOps for RBAC — all RBAC objects managed in Git, applied via automated pipelines (ArgoCD, Flux). Manual kubectl apply of RBAC is an anti-pattern in production. Git provides audit trail and review process.
11. Regular Access Review — quarterly review of all ClusterRoleBindings. Remove stale bindings. Rotate service account tokens. Review aggregated permissions as new CRDs are installed.
12. Namespace Isolation — each team/application gets its own namespace with ResourceQuota, LimitRange, and NetworkPolicy. Namespace admins can manage their resources but cannot see other namespaces.

9.2 The RBAC Design for TenantOPS Project

Here is how we would design RBAC for the TenantOPS project specifically:

Role	Subjects and Permissions
Platform Admin	platform-team group. ClusterRoleBinding to cluster-admin. Manages cluster infrastructure, RBAC, StorageClasses, nodes. Maximum 2-3 individuals.
Jenkins CI/CD	cicd-deployer ServiceAccount. Role in production/staging namespaces. Can update deployments, manage configmaps/services. Cannot read secrets or delete resources.
ArgoCD	argocd ServiceAccount. ClusterRole for reading all resources (needed for drift detection). Write access to all namespaces for sync.
Developers	dev-team group. RoleBinding to 'edit' ClusterRole in staging namespace. RoleBinding to 'view' ClusterRole in production namespace.

Role	Subjects and Permissions
Monitoring (Prometheus)	monitoring ServiceAccount. ClusterRole for reading metrics endpoints, nodes, pods across all namespaces.
Application Pods	Dedicated ServiceAccount per application. automountServiceAccountToken: false. No RBAC unless the app explicitly needs API access.

9.3 Namespace Isolation Architecture

```
# ResourceQuota for tenant namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-quota
  namespace: team-alpha
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    persistentvolumeclaims: "10"
    count/secrets: "20"
    count/configmaps: "50"
```

```
# LimitRange - enforce defaults
apiVersion: v1
kind: LimitRange
metadata:
  name: tenant-limits
  namespace: team-alpha
spec:
  limits:
    - type: Container
      default:
        cpu: 500m
        memory: 512Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        cpu: "4"
        memory: 4Gi
```

```
# NetworkPolicy - default deny all
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
```

```
namespace: team-alpha
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  # No ingress/egress rules = deny all
```

9.4 Real Attack Scenarios and Defenses

Scenario 1: Compromised Application Pod

Attack: An attacker exploits a vulnerability in the tenantops-api pod and gets shell access.

Without security hardening: The auto-mounted service account token has cluster-admin (a common misconfiguration). The attacker reads all secrets in all namespaces, finds AWS credentials, exfiltrates data.

With security hardening: automountServiceAccountToken: false on the service account. No token in the pod. Attacker cannot authenticate to the Kubernetes API. NetworkPolicy prevents egress to the API server. Blast radius limited to what that specific pod can access.

Scenario 2: Malicious RBAC Escalation

Attack: A developer in the staging namespace discovers they have the 'edit' ClusterRole via RoleBinding. They try to create a RoleBinding granting themselves cluster-admin.

Kubernetes defense: The edit ClusterRole does NOT include permissions on rbac.authorization.k8s.io resources. The developer cannot create, modify, or view RoleBindings. The escalation attempt returns 403 Forbidden.

Additional defense: OPA Gatekeeper policy requires all RoleBindings to be approved via GitOps PR process.

9.5 Production Security Checklist

- ✓ Never use cluster-admin service accounts for applications
- ✓ Set automountServiceAccountToken: false on all application service accounts
- ✓ Manage all RBAC via GitOps — no manual kubectl apply in production
- ✓ Regularly audit ClusterRoleBindings: kubectl get clusterrolebindings -o yaml
- ✓ Apply NetworkPolicies with default-deny in all namespaces
- ✓ Use Pod Security Standards (restricted) for application namespaces
- ✓ Never grant secrets read access without explicit justification
- ✓ Never grant pods/exec in production unless for specific debugging workflows
- ✓ Separate operator service accounts from application service accounts
- ✓ Implement OPA Gatekeeper or Kyverno for policy enforcement
- ✓ Enable Kubernetes audit logging — log all API server requests

- ✓ Use OIDC for human authentication — not static kubeconfigs
- ✓ Rotate all service account tokens regularly
- ✓ Review and limit wildcard usage: `kubectl get clusterroles -o yaml | grep "*"`
- ✓ Use IRSA for cloud provider access — never node-level IAM for pods

9.6 Final Interview Questions

Q: Design an RBAC architecture for a multi-tenant Kubernetes cluster where teams should not see each other's resources.

A: Namespace-per-tenant isolation. Each team gets: a dedicated namespace, ResourceQuota to limit resource usage, LimitRange for defaults, NetworkPolicy with default-deny, and a RoleBinding of the built-in 'admin' ClusterRole in their namespace (via namespace-specific RoleBinding, not ClusterRoleBinding — so they can only manage their namespace). Cluster-scoped resources (nodes, PVs, StorageClasses) accessible only via ClusterRole bound to platform team. All RBAC managed in GitOps. OPA Gatekeeper enforces that namespaced resources cannot reference resources in other namespaces.

Q: What is the risk of granting 'create pods' permission without 'get secrets' permission, and how do you mitigate it?

A: A user who can create pods can effectively read any secret in the namespace by mounting it as a volume or environment variable and then reading it via `kubectl logs` or a simple application in the pod. The permission to create pods is therefore nearly equivalent to secrets read. Mitigation: treat pod create permission as equivalent to secrets read in your security model. Use OPA Gatekeeper or Kyverno policies to restrict which images can run, validate pod specs against allowed configurations, and require all pods to come from CI/CD pipeline (not manual creation). Alternatively, use pod admission controls to strip sensitive env vars or volume mounts that were not approved.

Q: How does admission control complement RBAC? Can they replace each other?

A: RBAC controls WHO can perform WHAT action on WHICH resource type. Admission control controls WHAT CONTENT is acceptable in resources that are already authorized. They complement each other and cannot replace each other. Example: RBAC says 'developer can create pods'. Admission control then validates/mutates those pods: 'pods must have resource limits, must use images from approved registries, must have security context set.' RBAC alone cannot enforce content constraints — it only controls access. Admission control alone cannot enforce identity-based access — it sees every request after authorization.

Kubernetes RBAC Mastery — Security from Foundation to Enterprise

Least privilege. Zero trust. GitOps-managed. Production-hardened.